

COL7(6)83 Assignment 2

Due date: 5 October, 2025

This assignment deals with frequency-domain filtering, denoising, and deconvolution. All tasks have to be implemented yourself by working with the array of image pixels, unless otherwise specified in the problem. You are permitted to perform all tasks using intensity images.

I have provided some standard test images used in image processing. You may use these for your results.

PART 1

1. When zooming an image to a larger size, one only needs to perform interpolation, but when shrinking it to a smaller size, one also needs low-pass filtering in order to avoid aliasing. Let us study this in detail. Assume the original $M \times N$ image f was sampled with a spacing of ΔX samples per unit length in both directions, hence covers a physical size of $X = M\Delta X$ by $Y = N\Delta X$ units. Scaling the image size by a factor $k \in \mathbb{R}$ corresponds to resampling with a spacing of $\Delta X' = \Delta X/k$; the case $k > 1$ corresponds to zooming, $0 < k < 1$ corresponds to shrinking.
 - a. Suppose we obtain the zoomed/shrunk image f' simply by interpolating f at the new sample locations. What is the Fourier transform of f' in terms of the Fourier transform of f ? Here both f and f' should be treated as sampled signals i.e. sums of impulses.
 - b. In terms of the given parameters, particularly k , what is the exact form of the frequency-domain filter that should be applied before interpolation, so that the resulting f' is unaffected by aliasing? What is the corresponding kernel in the spatial domain?
 - c. Implement both variations of image resizing, i.e. the naive approach (using only interpolation), and your approach with a filtering step. The filter should be implemented in the frequency domain. Show the results on an image with significant high-frequency detail, such as the `barbara` test image, with different values of $k = 1/2, 1/4, 1/8$. To clearly see the effects, you may wish to zoom the image back to original size using nearest neighbour interpolation.

(Note: In practical implementations of image resizing, the filtering step is performed in the spatial domain with a kernel of small width for performance reasons. You are asked to implement an ideal version in the frequency domain as a pedagogical exercise.)

2. In an out-of-focus photograph, the point spread function (PSF) is typically a disk, because that is the shape of the camera's aperture. However, one can take photographs with uniquely shaped blurs by modifying the aperture shape! One can simulate this on an existing image by convolving it with a desired PSF.

- a. Choose an image h of size roughly 100-200 pixels in both directions, to use as a point spread function. Use intensity transformations if necessary to ensure that it consists of a bright shape on a black background. Using your resizing function from task 1, or a built-in function, create versions in different sizes, with the smallest being about 10 pixels in size, and normalize the intensities so they add up to 1.
- b. Perform blurring using convolution in the spatial domain, along with mirror padding. You may use a built-in convolution function, or your own implementation. Either way, take care that the kernel h is treated as properly centered, i.e. its origin should be in the center and not the top-left corner. Demonstrate the results using the smallest version of h on (i) an image that contains a few isolated impulses and is zero elsewhere, and (ii) a rectangular photograph of size on the order of 1000×2000 .
- c. Implement the same blurring via multiplication in the frequency domain. You may use a built-in FFT implementation. Make sure you get nearly the same result as in part (b), up to floating-point rounding error. Also show the intermediate images, as in lectures 9 and 10.
- d. Measure the compute time $T(r)$ of the implementations in (b) and (c) for all the different-sized versions of h . Plot the timings of both methods on the same log-log plot, with the size of the kernel $\max(m, n)$ on the x-axis.

(Note: For visually attractive results in (b) and (c), try using a photograph containing some very bright points, e.g. a scene at night, convert it to a datatype with higher precision, and perform a gamma transformation with $\gamma \approx 2.2$ before blurring and undo it afterwards.)

PART 2

The following three problems each have an optional task. If you are in a group of 2, you are required to complete *one* of the three optional tasks; if you are working individually, no optional task is required. In either case, doing one more optional task (on top of the 1 or 0 required) will give some extra marks; doing two more will not increase the marks further.

3. In printing, the appearance of continuous shades of grey is created by printing small dots of black ink; this process is called *halftone*. Many scans of such halftone prints are available on Wikimedia Commons. Take one of these images, making sure to choose one where the halftone dots are clearly visible. If necessary, you may crop it to a reasonable size for easier processing.

- a. Visualize the Fourier spectrum of the image, *without* padding, and with the DC component centered. Report the locations of the spikes corresponding to the halftone dot pattern, and comment on the relationship between these frequencies and the spacing of the dots in the spatial domain.
 - b. Using either a Gaussian or a Butterworth ($n = 2$) profile, construct a notch filter to remove these dots. Choose the width of the filter for best results, and show both the filtered frequency spectrum and the resulting image.
 - c. Using the same width, change the profile to an ideal filter (i.e. a hard cutoff in the frequency domain) and show the spectrum and the filtered image again.
 - d. **Optional:** Implement *optimum notch filtering* using the filter in (b) as a starting point. Be careful to choose a large enough neighbourhood size which spans at least a couple periods of the dot pattern. Show the estimated noise η obtained from the notch filter, the intermediate values $\text{cov}(g, \eta)$ and $\text{var}(\eta)$ and the computed weight w , as well as the final result. Make sure you perform your computations in a datatype with sufficient precision!
4. This problem is simply about evaluating some basic techniques' effectiveness at denoising on some synthetic test cases. We will use two test images: c being a constant image where each pixel has intensity $I_{\max}/2$, and f being a natural image of your choice.
- a. Create three noisy images from c , namely: c_1 with additive uniform noise, c_2 with additive Gaussian noise, and c_3 with salt-and-pepper noise (equal probability of salt and pepper). In each case, choose the parameters of the noise so that PSNR = 20 dB. Do the same starting from f .
 - b. Denoise each c_i with an arithmetic mean filter of width $w = 2a + 1$, and record the PSNR of the denoised image as a function of w . Do the same with a median filter, and make a single plot showing the PSNR of both methods with w on the x -axis. (You have get 3 plots total; one for each noise model.)
 - c. Repeat task (b) with the noisy natural images f_i . You should observe increase and then decrease with w ; explain in your report why this is the case. Also show the denoised images with the best PSNR for both mean and median filtering.
 - d. **Optional:** Implement a non-local means filter for higher-quality denoising, and demonstrate it on f_i . You may wish to refer to [the original paper](#) for details. To improve performance, only consider contributions from pixels within a $3m \times 3m$ "search window", where $m \times m$ is the neighbourhood size. Experiment with different values of both parameters (neighbourhood size m , standard deviation σ) for best results.
5. Take one of your results from problem 2 with a large amount of blur. Add Gaussian noise with PSNR = 20 dB as in 4(a) to obtain the degraded image g . We will attempt to perform

deconvolution and recover the original image f .

- a. Visualize the frequency bands of f as follows: Let $l_0, l_1, l_2, \dots$ be various low-pass filtered versions of f , with l_i containing only those frequency components with $u, v \in [-2^{i-1}, 2^{i-1}]$ (so l_0 contains only the DC component). Then compute the band-pass filtered images $b_i = f_i - f_{i-1}$ for $i = 1, 2, \dots$. Show $l_1, l_2, \dots$ and $b_1, b_2, \dots$, adding 128 to the b_i 's since they have both positive and negative values. Do the same for g and show the low-pass and band-pass filtered images.
- b. Apply an inverse filter in the frequency domain to get a (poorly) restored image. Show its frequency bands as in (a) and comment on how well different frequency bands are restored.
- c. To apply a Wiener filter, you will need to estimate $S_\eta(u, v)$ and $S_f(u, v)$. Making the assumption that $\eta[x, y]$ has zero mean and that $|N(u, v)|$ is constant, use the Plancherel theorem to find $S_\eta(u, v)$ in terms of the noise variance σ . Using the same logic, find $S_f(u, v)$ in terms of $\|f\|$ assuming that $|F(u, v)|$ is constant.
- d. Perform Wiener filtering assuming that S_η/S_f is a constant K , and plot the PSNR of the restored image $\hat{f}$ (i.e. using $\|f - \hat{f}\|$ in the denominator) for a range of values of K . On the plot, mark the value of K that corresponds to the estimates of S_η and S_f obtained in (c). Show the restored image which has the highest PSNR, and also its frequency bands as in (a) and (b).
- e. So far we have used a degraded image g with a small amount of noise (PSNR 20 dB). Repeat (d) with even less noise (PSNR 30 dB) and with more noise (PSNR 10 dB) and show the results.
- f. **Optional:** Work out how to perform regularized deconvolution when the regularizer is the squared gradient norm, $R(\hat{f}) = \sum_{x,y} ((\hat{f}(x+1, y) - \hat{f}(x))^2 + (\hat{f}(x, y+1) - \hat{f}(x, y))^2)$. You will have to evaluate $R(\hat{f})$ in terms of the Fourier transform $\hat{F}$, and then find a closed-form expression for the optimal $\hat{F}$ similar to the derivation of the Wiener filter discussed in class. As in the previous tasks, plot the PSNR for a range of values of the regularization parameter λ , and show the best resulting image.

SUBMISSION

You will submit a zip file that contains (i) all your code for the assignment, (ii) a PDF report that includes your results for each of the assignment problems. In the text of the report, also provide all relevant information for each part.

All images should ideally be saved in PNG format, which is lossless and so does not cause any information loss (other than quantization if the intensities are not already in 8-bit format). JPEG is permitted if your submission PDF is becoming too big for the upload limit.

Your assignment should be submitted on Moodle (old), before midnight on the due date. Only one person in a group needs to submit. Late days are counted with a quantization of 0.5 days: if you cannot finish the assignment by midnight, get some sleep and submit by noon the following day.

Separately, each of you must individually submit a short response in a separate form regarding how much you and your partner contributed to the work in this assignment. Your assignment marks will not be released until you submit the form.